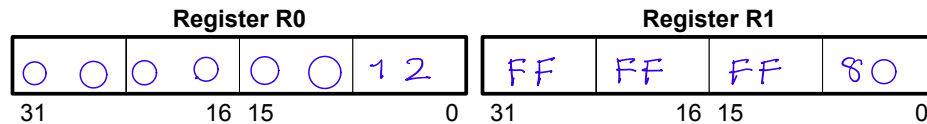


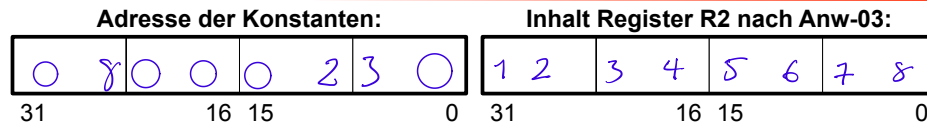
1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.

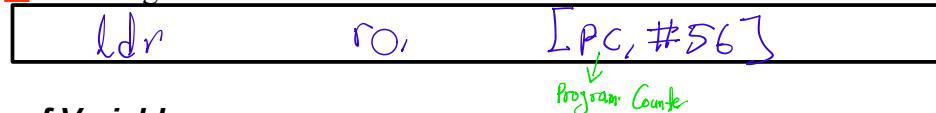
Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:



Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:



Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

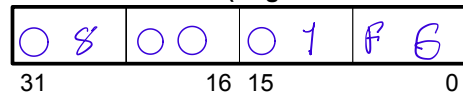


2 Zugriff auf Variable

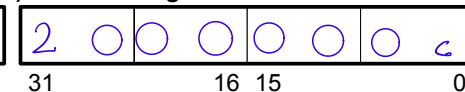
Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss

Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:

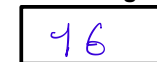
Adresse v. VariableA (Reg. R0 nach Anw-05)



Inhalt Register R1 nach Anw-05:

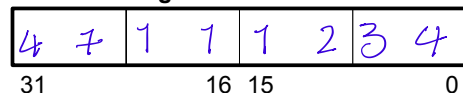


Anzahl von Anw-05 geladener Bits:

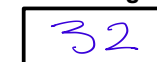


Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

Inhalt Register R2 nach Anw-06:

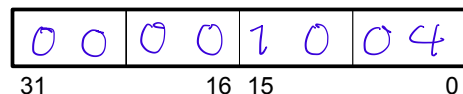


Anzahl von Anw-06 geladener Bits:

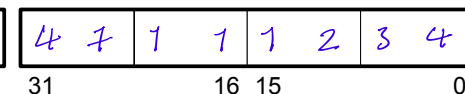


Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

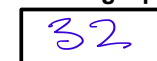
Adresse v. VariableC:



Inhalt von VariableC nach Anw-07:



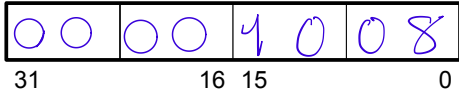
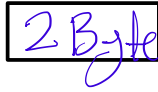
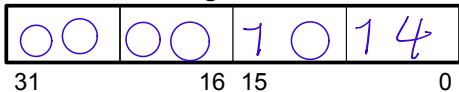
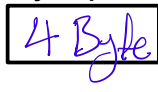
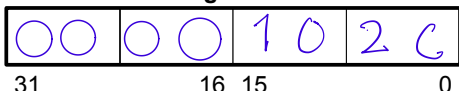
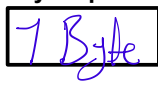
Anzahl von Anw-07 gespeicherter Bits:



3 Zugriff auf Felder

$$24 = 16 + 8 = 0x0018 = 0x18$$

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

MeinHalbwortFeld:	Anfangsadresse: 	Anzahl Elemente: 	Anzahl Bytes pro Element: 
MeinWortFeld:	Anfangsadresse: 	Anzahl Elemente: 	Anzahl Bytes pro Element: 
MeinTextFeld:	Anfangsadresse: 	Anzahl Elemente: 	Anzahl Bytes pro Element: 

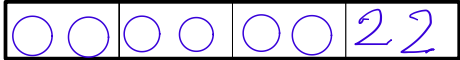
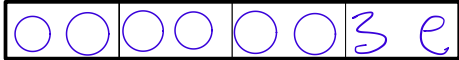
$$0x1014 + 0x0018 = 0x102C$$

Anw-08

Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für das jeweilige Element erfolgen.

Anw-09

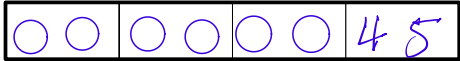
Anw-10

Register R1 nach Anw-09:	Register R2 nach Anw-10:
	
31 16 15 0	31 16 15 0

Anw-11

Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:

Anw-12

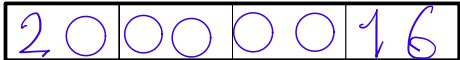
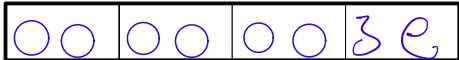
Register R4 nach Anw-12:

31 16 15 0

Nr. (beginnend mit 0) des von Anw-12 gelesenen Elements:



Anw-13

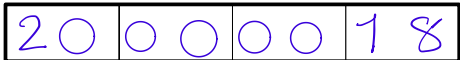
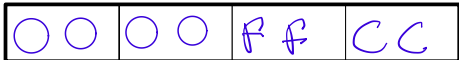
Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

Register R0 nach Anw-13	Register R5 nach Anw-13:
	
31 16 15 0	31 16 15 0

Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements:



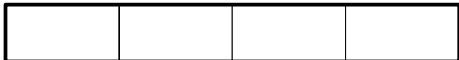
Anw-14

Register R0 nach Anw-14	Register R6 nach Anw-14:
	
31 16 15 0	31 16 15 0

Nr. (beginnend mit 0) des von Anw-14 gelesenen Elements:



Anw-15

Register R0 nach Anw-15	Adresse des von Anw-15 überschriebenen Elements:
	
31 16 15 0	31 16 15 0

Nr. (beginnend mit 0) des von Anw-15 überschriebenen Elements:



4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

Addition

Anw-16
Anw-17
Anw-18
Anw-19

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R1:	000100100001101000101011001111000	00305419896	00305419896
Reg. R2:	10011101110010100101100110000110	-1647687290	02647280006
Reg. R3:	10101111111111101010111111111110	-1342259202	02952708094
31 16 15 0 Kreuzen Sie die gesetzten Bits des Statusregisters an: V C ☒ N Z		Ergebnis: ☒ Ok ☐ Nicht Ok	Ergebnis: ☒ Ok ☐ Nicht Ok

Subtraktion

Anw-20
Anw-21
Anw-22

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R4:	11001100000000000000000000000000	-872475232	3422552064
Reg. R5:	01001110000000000000000000000000	1308622848	1306822848
Reg. R6:	01111110000000000000000000000000	2113929216	2113929216
31 16 15 0 Kreuzen Sie die gesetzten Bits des Statusregisters an: ☒ V C ☐ N Z		Ergebnis: ☐ Ok ☒ Nicht Ok	Ergebnis: ☒ Ok ☐ Nicht Ok

Anw-23
Anw-24
Anw-25

		dezimale Interpretation der Bitmuster	
		signed	unsigned
Reg. R7:	00100111000000000000000000000000	654317424	654317424
Reg. R8:	01000101000000000000000000000000	1157627904	1157627904
Reg. R9:	11100010000000000000000000000000	-503376480	3791650876
31 16 15 0 Kreuzen Sie die gesetzten Bits des Statusregisters an: V C ☒ N Z		Ergebnis: ☒ Ok ☐ Nicht Ok	Ergebnis: ☐ Ok ☒ Nicht Ok